Unification of Higher-Order Patterns in a Simply Typed Lambda-Calculus with Finite Products and Terminal Type

Roland Fettig and Bernd Löchner

Universität Kaiserslautern, Fachbereich Informatik, Postfach 3049,
D-67653 Kaiserslautern, Germany
{fettig,loechner}@informatik.uni-kl.de

Abstract. We develop a higher-order unification algorithm for a restricted class of simply typed lambda terms with function space and product type constructors. It is based on an inference system manipulating so called higher-order product-patterns which is proven to be sound and complete. Allowing tuple constructors in lambda binders provides elegant notations. We show that our algorithm terminates on each input and produces a most general unifier if one exists. The approach also extends smoothly to a calculus with terminal type.

1 Introduction

Functional programming languages based on typed lambda calculus like *ML* or *Haskell* are usually equipped with type constructors for cartesian products besides the function space constructor. This offers the possibility of modelling and processing complex data structures in a strictly typed framework where type checking can avoid many programming errors. Moreover, the tupling of values is a very successful strategy for improving the efficiency of recursive functions due to the availability of all intermediate results needed by the computation.

On the other hand, higher-order rewriting covers a large part of typed functional languages and demonstrates its computational power in the context of equational reasoning on functional programs. Typical applications are partial evaluation and program transformation. This kind of reasoning requires the solution of complex higher-order unification problems, which is demonstrated by a case study on the transformation of a program modifying a state monad [Joh94]. However, reasonable unification algorithms for calculi containing product types are not yet available. It turns out that one has to put a lot of effort into the solution of problems introduced by the presence of projection functions.

Example 1. Consider the following set of equations: $\{\pi_1^2(x) =^? \pi_1^2(s),\ \pi_2^2(x) =^? \pi_2^2(t)\}$, where x is a variable of product type and s, t are ground terms. Neither $\theta_s = \{x \leftarrow s\}$ nor $\theta_t = \{x \leftarrow t\}$ unifies both equations. Nevertheless, the unification problem has the (unique) solution $\theta = \{x \leftarrow \langle \pi_1^2(s), \pi_2^2(t) \rangle\}$.

In this paper we propose an inference system for higher-order unification of simply typed lambda terms with function space and product type constructors. It is based on a calculus with extended lambda abstractions. In general,

higher-order unification is undecidable [Gol81], even if product types are abandoned. We therefore describe a decidable subsystem by generalizing the concept of higher-order patterns – introduced by Miller [Mil91] and reformulated by Nipkow [Nip91] – to so called *higher-order product-patterns*. For this class of terms a sound and complete unification algorithm is presented. Moreover, we show that our algorithm terminates on each input and produces a most general unifier if one exists. Finally, we outline an extension to integrate the so called terminal type.

2 Types, Terms and Substitutions

Since we want to distinguish between objects being tuples and objects being functions it is necessary to describe what kind of types we are dealing with. Additionally, we have to lay down the way objects of certain types are created.

2.1 Type constructors

Besides the usual function space constructor we introduce a type constructor $\times_n$ for each $n \geq 2$ describing finite products of arity n. Because the arity of a finite product can always be inferred from the context, we omit the subscript of $\times$.

Definition 1 (Types). Let $\mathcal{B}$ be a denumerable set of base types. The set of *types* $\mathcal{T}(\mathcal{B})$ generated by function space ($\rightarrow$) and finite product ($\times$) type constructors is defined by the following grammar. Here B ranges over base types.

$$T \quad = \quad B \mid T \rightarrow T \mid T \times \cdots \times T$$

Note that several isomorphisms can be described in $\mathcal{T}(\mathcal{B})$ (c. f. [DC95]). But, even if e. g. the types $A \rightarrow (B \times C)$ and $(A \rightarrow B) \times (A \rightarrow C)$ are isomorphic, we do not identify them, nor any others.

2.2 Term constructors and Typing rules

Abstraction in lambda calculus binds variables in λ-expressions. Product types lead to a more general way of abstraction, namely binding of tuples of variables.

Definition 2 (Abstraction Patterns). Let $\mathcal{V}$ be a denumerable set of variables. The set of *abstraction patterns* $\mathcal{P}(\mathcal{V})$ is defined to be the set of all linear terms generated by the following grammar. Here V ranges over variables.

$$P \quad = \quad V \mid \langle P, \ldots, P \rangle$$

Note that abstraction patterns necessarily have to be linear in order to permit unconstrained β-reduction. For example, the term $((\lambda\langle x, x\rangle.t)\ s)$ would only be reducible if s were equivalent to $\langle \hat{s}, \hat{s} \rangle$.

Definition 3 (Terms). Let $\mathcal{V}$ be a denumerable set of variables and $\mathcal{F}$ be a denumerable set of constants. The set of *untyped terms* $Term(\mathcal{F}, \mathcal{V})$ is defined by the following grammar. Here V ranges over Variables, F ranges over constants, and P ranges over abstraction patterns.

$$T \quad = \quad V \mid F \mid \lambda P.T \mid (T\ T) \mid \langle T, \ldots, T \rangle \mid \pi_i^n(T)$$

$$\frac{sig(a) = \tau}{\Gamma \vdash a : \tau}\ const \qquad \frac{}{\Gamma, x : \tau \vdash x : \tau}\ var$$

$$\frac{\Gamma, p_1 : \tau_1, \ldots, p_n : \tau_n \vdash t : \tau}{\Gamma, \langle p_1, \ldots, p_n \rangle : \tau_1 \times \cdots \times \tau_n \vdash t : \tau}\ \times pattern$$

$$\frac{\Gamma \vdash t : \tau_1 \times \cdots \times \tau_n}{\Gamma \vdash \pi_i^n(t) : \tau_i}\ \times elim \qquad \frac{\Gamma \vdash t_1 : \tau_1 \quad \cdots \quad \Gamma \vdash t_n : \tau_n}{\Gamma \vdash \langle t_1, \ldots, t_n \rangle : \tau_1 \times \cdots \times \tau_n}\ \times intro$$

$$\frac{\Gamma \vdash s : \tau \to \hat{\tau} \quad \Gamma \vdash t : \tau}{\Gamma \vdash (s\ t) : \hat{\tau}}\ \to elim \qquad \frac{\Gamma, p : \tau \vdash t : \hat{\tau}}{\Gamma \vdash \lambda p.t : \tau \to \hat{\tau}}\ \to intro$$

The notation $\Gamma, p : \tau$ is used as an abbreviation for $\Gamma \cup \{p : \tau\}$. All inference rules are supposed to respect linearity of abstraction patterns and environments.

Fig. 1. Typing rules

A sequence of terms is denoted by $\overline{t_m}$ where m is the number of terms in such a sequence. Sequences of variables are denoted by $\overline{x_m}$ and sequences of abstraction patterns are denoted by $\overleftrightarrow{u_m}$. We also use the notation $f(\overline{t_m})$ as an abbreviation for $(\ldots(f\ t_1)\ldots\ t_m)$ and the notation $\lambda\overleftrightarrow{u_k}.\,t$ as an abbreviation for $\lambda u_1.\ \ldots\ \lambda u_k.\,t$. The set of variables of an abstraction pattern p is denoted by $Var(p)$. Occurrences of variables in a term are distinguished as being free or bound (by a surrounding lambda abstraction). Hence, the set of free (resp. bound) variables of a term t is denoted by $FVar(t)$ (resp. $BVar(t)$). These notions also apply to sequences and sets of terms.

Definition 4 (Signature and Typing). A *signature sig* is a mapping from the set of constants $\mathcal{F}$ to the set of types $\mathcal{T}(\mathcal{B})$. A set Γ of statements of the form $p : \tau$ where p is an abstraction pattern and τ is a type is called an *environment* if it is linear, i. e. each variable x appears at most once in Γ.
A statement of the form $\Gamma \vdash t : \tau$ where Γ is an environment, t is a term, τ is a type is called a *typing judgement* if it is generated by the typing rules in Fig. 1.
If $\Gamma \vdash t : \tau$ is a typing judgement then τ is called the *type of* t *under* Γ.
If there is a type τ of t under Γ then t is called *typable under* Γ.

The linearity of Γ guarantees that each variable x typable under Γ has a unique type τ_x. Therefore, terms typable under Γ have unique types determined by Γ (assuming that abstraction patterns in lambda expressions are decorated with their types). Moreover, it is well known that typability is decidable. In the remaining parts of the paper we assume that all terms introduced are typable under a fixed environment Γ. Accordingly, the type of each term is fixed by Γ.

2.3 Typed Substitutions

A substitution in a typed lambda calculus framework has to be typed, i. e. a term substituted for a variable has to have the same type as the variable.

Definition 5 (Substitution). Let Γ be an environment. A mapping θ from $\mathcal{V}$ to $Term(\mathcal{F}, \mathcal{V})$ is called a Γ-substitution if $\Gamma \vdash \theta(x) : \tau$ for each $x : \tau \in \Gamma$.

Substitutions are lifted to terms by replacing all free variables by its substitutes and renaming bound variables in order to avoid accidental capturing. Substitutions are applied to sequences and sets of terms in the same way. We will use some standard notions for handling substitutions summarized below.

Composition of substitutions is usual function composition. The domain $Dom(\theta)$ is the set of variables x, s.t. $\theta(x) \neq x$. The image $Im(\theta)$ is the set of terms $\theta(x)$, s.t. $\theta(x) \neq x$. $\mathcal{I}(\theta) = FVar(Im(\theta))$ is the set of free variables in the image of θ. If $Dom(\theta_1) \cap Dom(\theta_2) = \varnothing$ then their union $\theta_1 \cup \theta_2$ is defined to be a substitution which applies both parts in parallel.

The restriction of a substitution θ to a set of variables V is a substitution denoted by $\theta\big|_V$ and defined as $\theta\big|_V(x) = \theta(x)$ for each $x \in Dom(\theta\big|_V) = Dom(\theta) \cap V$. For comparing substitutions we use a subsumption ordering which can be parameterized by any equivalence relation $=_\omega$ on terms which is stable w.r.t. substitutions. Thus, $\theta \geq_\omega \sigma$ holds if there is a substitution ρ s.t. $\theta(t) =_\omega (\rho \circ \sigma)(t)$ for each typable term t. The concept of subsumption ordering is further generalized by restricting the domains of the substitutions compared, i.e. $\theta \geq_\omega \sigma\,[V]$ holds if $\theta\big|_V \geq_\omega \sigma\big|_V$.

A substitution θ is called ω-idempotent w.r.t. a stable congruence relation $=_\omega$ if $\theta \circ \theta =_\omega \theta$. It is called ω-normalized w.r.t. a normalizing rewrite relation $\rightarrow_\omega$ if $\theta(x)$ is in ω-normal form for each $x \in Dom(\theta)$.

3 Equational Theory

Lambda calculus comes equipped with two congruence relations called β- and η-conversions. Introducing these relations as well as their counterparts for terms of product type and analyzing their properties is the purpose of this section.

Note that all conversions of our calculus will be applicable to any (typable) term of appropriate structure, like in ordinary lambda calculus. For β-conversion the existence of a matching substitution has to be established. Besides providing linear abstraction patterns only, this is guaranteed by allowing sp-equivalence in the condition for elementary β-reduction instead of requiring syntactic identity.

Definition 6 (congruence relations). For $\omega \in \{\alpha, \beta, \eta, \lambda, \pi, sp, \lambda\times\}$ let $=_\omega$ be the smallest congruence relation on typable terms containing the elementary relation $\rightarrow^\epsilon_\omega$ defined as follows.

$$\begin{array}{lll}
\alpha: & \lambda p.t \rightarrow^\epsilon_\alpha \lambda q.\theta(t) & \text{if } Dom(\theta) \subseteq Var(p) \wedge \theta(p) = q \wedge \mathcal{I}(\theta) \cap FVar(t) = \varnothing \\
\beta: & ((\lambda p.t)\, s) \rightarrow^\epsilon_\beta \theta(t) & \text{if } Dom(\theta) \subseteq Var(p) \wedge \theta(p) =_{sp} s \\
\eta: & \lambda p.(t\, p) \rightarrow^\epsilon_\eta t & \text{if } Var(p) \cap FVar(t) = \varnothing \\
\pi: & \pi^n_i(\langle t_1, \ldots, t_i, \ldots, t_n\rangle) \rightarrow^\epsilon_\pi t_i & \lambda: \rightarrow^\epsilon_\lambda \;=\; \rightarrow^\epsilon_\alpha \cup \rightarrow^\epsilon_\beta \cup \rightarrow^\epsilon_\eta \\
sp: & \langle \pi^n_1(t), \ldots, \pi^n_n(t)\rangle \rightarrow^\epsilon_{sp} t & \lambda\times: \rightarrow^\epsilon_{\lambda\times} \;=\; \rightarrow^\epsilon_\lambda \cup \rightarrow^\epsilon_\pi \cup \rightarrow^\epsilon_{sp}
\end{array}$$

All elementary relations $\rightarrow^\epsilon_\omega$ are decidable. While $\rightarrow^\epsilon_\alpha$ requires syntactic (first order) matching of abstraction patterns, which is trivial, $\rightarrow^\epsilon_\beta$ requires matching an abstraction pattern against an arbitrary term modulo sp-equivalence. Since

abstraction patterns are defined to be π_i^n-free, finitely many *sp*-expansions can transform this matching problem into a syntactic one, the decidability of which is trivial again.

Example 2. Let x be of type $A \times B$. Then $\lambda x.\pi_1^2(x) \to_\alpha^\epsilon \lambda\langle y,z\rangle.\pi_1^2(\langle y,z\rangle) =_\pi \lambda\langle y,z\rangle.y$. Moreover, $((\lambda\langle y,z\rangle.y)\, s) \to_\beta^\epsilon \pi_1^2(s)$ for any term s of type $A \times B$.

The usual $\alpha\beta\eta$-conversions of simply typed lambda calculus without product types are contained in $=_\lambda$ as special cases. Compatibility with substitutions carries over to the calculus with product types: All relations of Def. 3 have the property of being stable w. r. t. substitutions. This is very important for the solution of unification problems modulo a congruence relation since terms can be restricted to be appropriate representatives of their congruence classes. The question is, what do appropriate representatives for higher-order unification modulo $=_{\lambda\times}$ look like. For higher-order unification in the simply typed lambda calculus without product types those representatives usually are the 'long $\beta\eta$-normal forms' introduced by Huet [Hue75]. We generalize this concept here in order to obtain 'long $\lambda\times$-normal forms'.

Definition 7 (α-conversion). Let $\geq_\alpha$ be the smallest quasi ordering on typable terms containing the relation $\to_\alpha^\epsilon$ and closed under context embedding. The partial ordering induced by $\geq_\alpha$ is defined as $\to_\alpha \;=\; \geq_\alpha \setminus {}_\alpha\!\leq$. The equivalence relation induced by $\geq_\alpha$ is defined as $\approx_\alpha \;=\; \geq_\alpha \setminus \to_\alpha$.

The variable renaming part of α-conversion is now denoted by $\approx_\alpha$ whereas $\to_\alpha$ additionally introduces structural information into abstraction patterns. There are $\to_\alpha$-irreducible terms because the set of abstraction patterns of the same type is finite (up to variable renaming). Moreover, it is easy to prove that $\to_\alpha$ is strongly normalizing and confluent modulo $\approx_\alpha$. The corresponding α-normal form of a term t is denoted by $t{\downarrow_\alpha}$.

There are several ways of handling the extensional conversions. An overview of their advantages and weaknesses can be found in [JG95]. Here we introduce contractions as well as restricted expansions.

Definition 8 (extensional reductions). The extensional rewrite relations $\to_\eta$, $\twoheadrightarrow_\eta$, $\to_{sp}$, and $\twoheadrightarrow_{sp}$ on typable terms are defined as follows.

$t \twoheadrightarrow_\eta^\epsilon \lambda p.(t\, p)$ if $\lambda p.(t\, p) \to_\eta^\epsilon t$ and $t \neq \lambda q.\hat{t}$
The *η-contraction* $\to_\eta$ is the smallest relation containing $\to_\eta^\epsilon$ which is closed under context embedding. The *restricted η-expansion* $\twoheadrightarrow_\eta$ is the smallest relation containing $\twoheadrightarrow_\eta^\epsilon$ which is closed under context embedding except for applications, i. e. $(t\, s) \not\twoheadrightarrow_\eta ((\lambda p.(t\, p))\, s)$.

$t \twoheadrightarrow_{sp}^\epsilon \langle \pi_1^n(t), \ldots, \pi_n^n(t)\rangle$ if $\langle \pi_1^n(t), \ldots, \pi_n^n(t)\rangle \to_{sp}^\epsilon t$ and $t \neq \langle t_1, \ldots, t_n\rangle$
The *sp-contraction* $\to_{sp}$ is the smallest relation containing $\to_{sp}^\epsilon$ which is closed under context embedding. The *restricted sp-expansion* $\twoheadrightarrow_{sp}$ is the smallest relation containing $\twoheadrightarrow_{sp}^\epsilon$ which is closed under context embedding except for projections, i. e. $\pi_i^n(t) \not\twoheadrightarrow_{sp} \pi_i^n\,(\langle \pi_1^n(t), \ldots, \pi_n^n(t)\rangle)$.

Note that both $\twoheadrightarrow_\eta$ and $\twoheadrightarrow_{sp}$ are strongly normalizing due to the restrictions imposed but, that only $\twoheadrightarrow_{sp}$ is confluent. However, each diverging situation of the form $\lambda p.(t\,p)\ {}_\eta\!\!\leftarrow\ t \twoheadrightarrow_\eta \lambda q.(t\,q)$ can be closed using $\geq_\alpha$. Because p and q are both linear abstraction patterns of the same type, they are unifiable. Therefore we know that $\lambda p.(t\,p) \geq_\alpha \lambda o.(t\,o)\ {}_\alpha\!\leq \lambda q.(t\,q)$ where o is some abstraction pattern more specific than p and q.

Definition 9 (computational reductions). The computational rewrite relations $\rightarrow_\beta$ and $\rightarrow_\pi$ on typable terms are defined as the smallest extensions of their corresponding elementary relations which are closed under context embedding. Finally, $\twoheadrightarrow_\lambda$ and $\twoheadrightarrow_{\lambda\times}$ are defined as follows.

$$\twoheadrightarrow_\lambda \;=\; \rightarrow_\alpha \cup \rightarrow_\beta \cup \twoheadrightarrow_\eta \qquad\qquad \twoheadrightarrow_{\lambda\times} \;=\; \twoheadrightarrow_\lambda \cup \rightarrow_\pi \cup \twoheadrightarrow_{sp}$$

Combinations of several rewrite relations other than $\rightarrow_\lambda$ and $\rightarrow_{\lambda\times}$ are denoted by a sequence of their corresponding subscripts, e. g. $\rightarrow_{\eta\,sp} = \rightarrow_\eta \cup \rightarrow_{sp}$. The reflexive, transitive (and symmetric, resp.) closure of a rewrite relation $\rightarrow_\omega$ is denoted by $\overset{*}{\rightarrow}_\omega$ ($\overset{*}{\leftrightarrow}_\omega$, resp.).

Lemma 10. $=_{\lambda\times} \;=\; \overset{*}{\twoheadleftarrow}_{\lambda\times}$

Proof. $\overset{*}{\twoheadleftarrow}_{\lambda\times} \subseteq =_{\lambda\times}$ is trivial. $=_{\lambda\times} \subseteq \overset{*}{\twoheadleftarrow}_{\lambda\times}$ is shown by simulating the missing $=_\eta$- and $=_{sp}$-steps by appropriate $\overset{*}{\twoheadleftarrow}_{\alpha\beta}$- and $\overset{*}{\twoheadleftarrow}_\pi$-steps.

Note that the proof of Lem. 10 also establishes stability of $\overset{*}{\twoheadrightarrow}_{\lambda\times}$ w. r. t. context embedding and substitutions despite the fact that $\twoheadrightarrow_{\lambda\times}$ lacks both properties.

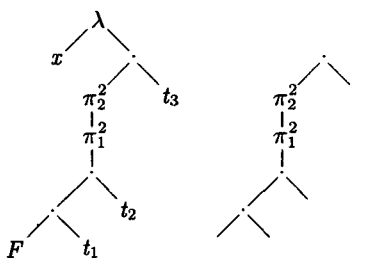
Lemma 11 (termination and confluence).
The rewrite relation $\twoheadrightarrow_{\lambda\times}$ is confluent modulo $\approx_\alpha$ and strongly normalizing.

Proof. Confluence is proved along the lines of Kesner given in [Kes94] for a similar calculus. The only difference besides not dealing with fixpoint types is in our definition of η- and α-conversion. The additional divergences produced by $\twoheadrightarrow_\eta$ can be closed by $\rightarrow_\alpha$ as discussed above. In order to prove strong normalization we use an argument of reduction. It is well known that the restriction of $\twoheadrightarrow_{\lambda\times}$ to terms which only abstract over variables is strongly normalizing, see e. g. [DCK93, Aka93, JG95]. Suppose that there were an infinite chain in $\twoheadrightarrow_{\lambda\times}$. Then, using $=_{\alpha\pi}$, it is easy to construct a similar infinite chain consisting of terms which do not contain any product-pattern. This contradicts the strong normalization of the restricted relation and we are done.

The unique (up to α-equivalence) $\lambda\times$-normal form of a term t is denoted by $t{\downarrow}_{\lambda\times}$ and is called *long $\lambda\times$-normal form* of t, as usual.

Theorem 12. *It is decidable whether two terms are $\lambda\times$-equivalent.*

Proof. By Lemmas 10, 11 and decidability of $\approx_\alpha$.



$t : \tau \to \hat{\tau}$
$t = \lambda x.(\pi_2^2(\pi_1^2((F\ t_1)\ t_2))\ t_3)$
$= \lambda x.\mathfrak{B}(F)(\overline{t_3})$
with $\mathfrak{B} = [1,(2,2),0,(1,2),2]$
and $F : \tau_1 \to \tau_2 \to ((\tau' \times (\tau_3 \to \hat{\tau})) \times \tau'')$

Fig. 2. The backbone of a term

4 Unification

We assume the reader to be familiar with some basic concepts of higher-order unification, see e. g. [SG89]. Please remember that if S is in solved form, the induced substitution σ_S is a most general unifier of S.

Since the seminal work of Huet [Hue75], one differentiates between flex and rigid terms. A substitution may alter the whole term structure of a flex term but only the arguments of a rigid term. In our extended calculus, these notions have to be generalized. We therefore introduce the concept of a *backbone* of a term, which is illustrated by an example in Fig. 2.

Definition 13. The *backbone* of a term $\lambda\overleftrightarrow{x_k}.(\pi_{i_j}^{n_j}(\cdots(\pi_{i_1}^{n_1}(a(\overline{t_{m_0}}))(\overline{t_{m_1}}))\cdots))(\overline{t_{m_j}})$ denotes the sequence of applications and projections and may be described by a sequence $[m_j,(i_j,n_j),\ldots,m_1,(i_1,n_1),m_0]$ of naturals to count the number of successive applications and pairs of naturals to identify a specific π-symbol, respectively. We introduce a shorthand notation $\lambda\overleftrightarrow{x_k}.\mathfrak{B}(a)(\overline{t_{m_0}},\overline{t_{m_1}},\ldots,\overline{t_{m_j}})$ where $\mathfrak{B}$ is a meta-variable representing the backbone.
A term $\lambda\overleftrightarrow{x_k}.\mathfrak{B}(a)(\overline{t_m})$ is called an *xflex term* if its *head* a is a free variable, otherwise it is called an *xrigid term*. The terms $\overline{t_m}$ are the *arguments* of a.

Note that these concepts truly generalize the ones used by Huet. A flex/rigid term is an xflex/xrigid term not containing any π-symbol in its backbone.[1]

4.1 Introducing Higher-Order Patterns

It is well known that the unification problem for the simply typed lambda calculus is undecidable in general [Gol81]. But by analyzing a restricted form of β-reduction, Miller [Mil91] discovered a class of terms, so called higher-order patterns, which have an 'easy', i. e. decidable and unitary, unification problem.

[1] Note that Duggan introduces in [Dug93] the concept of a 'locator' which somehow serves the same purpose than our backbone. However, his locators are concrete syntax, whereas backbones are metasymbols describing the structure of terms.

The essential property of higher-order patterns is that substituting free variables can only result in β_0-normalizable terms. This relation is very restrictive, since it only consists of steps of the form $((\lambda x.t)\,x) \to_{\beta_0} t$ (modulo implicit α-conversions). Hence, the implicit reduction steps following a substitution can not alter the term introduced by the substitution. Nipkow reformulated the approach in the context of higher-order rewriting [Nip91].

We introduce an adequate normal form to get a nice generalization of this concept. Note that $\to_{\eta\,sp}$ is confluent and strongly normalizing [Pot81]. This normal form permits an elegant definition of a higher-order product-pattern.

Definition 14. The *pattern normal form* $t{\downarrow_P}$ of a term t is the term which results from $t{\overleftrightarrow{\downarrow}_{\lambda\times}}$ by replacing the arguments of free variables with their $\to_{\eta\,sp}$ normal forms.

Definition 15. A *higher-order product-pattern* is a term t, where in $t{\downarrow_P}$ the arguments of free variables are restricted to be abstraction patterns of bound variables and in each xflex sub-term $\mathfrak{B}(F)(\overleftrightarrow{u_n})$ the sequence of abstraction patterns $\overleftrightarrow{u_n}$ is linear.

Example 3. Product-patterns[2] are e. g. $\lambda\langle x,y\rangle.F\langle y,x\rangle$, $((\lambda x.f\langle g\,x,(\lambda y.y)\,x\rangle)\,Z)$, or $t = \lambda u.((\pi_2^2\pi_1^2((F\,\pi_1^2\pi_2^2 u)\,\pi_1^2 u))\;\pi_2^2\pi_2^2 u)$, since its pattern normal form is $t{\downarrow_P} = \lambda\langle x,\langle y,z\rangle\rangle.((\pi_2^2\pi_1^2((F\,y)\,x))\,z) = \lambda\langle x,\langle y,z\rangle\rangle.\mathfrak{B}(F)(y,x,z)$. Non-patterns are e. g. $\lambda x.F(G(x))$ or $s = \lambda x.((F\,x)\,\pi_1^2(x))$, since $s{\downarrow_P} = \lambda\langle y,z\rangle.((F\,\langle y,z\rangle)\,y)$.

Please note that higher-order product-patterns are closed under $=_{\lambda\times}$ and pattern-substitutions. In the sequel we will only consider higher-order product-patterns which are implicitly converted to their pattern normal form. Furthermore, substitutions are supposed to be $\lambda\times$-idempotent and $\downarrow_P$-normalized.

4.2 Unification by Transformation

A very successful approach to describe unification methods is the use of inferences, which transform systems of equations into equivalent solved forms.

In contrast to [SG89] we explicitly distinguish the solved part S in the inferences of $\mathcal{PU}$, which are given in Fig. 3. This eases termination considerations. We write $(E,S) \vdash_{\mathcal{PU}} (E',S')$ for the transformation of E and S to E' and S' by use of an inference of system $\mathcal{PU}$.

There are two important invariants we have to mention: First, $\approx_\alpha$-conversions and $\downarrow_P$ normalizations are performed implicitly. Second, since substitutions may not alter the type of a term, both terms of an equation must have identical types. This ensures an appropriate typing of the newly introduced variables H.

One may distinguish two groups of inferences: The first four resemble first-order unification. The latter four handle free higher-order variables by some approximation technique. This will influence the structure of our proofs. The inferences of each group behave similarly.

[2] Uppercase letters denote free variables.

Delete:

$$\frac{\{s =^? s\} \cup E, S}{E, S}$$

Decompose:

$$\frac{\{\lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{s_n}) =^? \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_n})\} \cup E, S}{\{\overline{\lambda\overrightarrow{x_k}.s_n =^? \lambda\overrightarrow{x_k}.t_n}\} \cup E, S}$$

if $a \in \mathcal{F} \cup Var(\overrightarrow{x_k})$.

Product-Decompose:

$$\frac{\{\lambda\overrightarrow{x_k}.\langle\overrightarrow{s_n}\rangle =^? \lambda\overrightarrow{x_k}.\langle\overrightarrow{t_n}\rangle\} \cup E, S}{\{\overline{\lambda\overrightarrow{x_k}.s_n =^? \lambda\overrightarrow{x_k}.t_n}\} \cup E, S}$$

Eliminate:

$$\frac{\{\lambda\overrightarrow{x_k}.F(\overrightarrow{x_k}) =^? t\} \cup E, S}{\sigma(E), \sigma(S) \cup \sigma}$$

if $F \in \mathcal{V} - Var(\overrightarrow{x_k})$, $F \notin FVar(t)$, and $\sigma = \{F \leftarrow t\}$.

flex-xrigid:

$$\frac{\{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_m})\} \cup E, S}{\{F =^? \lambda\overrightarrow{u_n}.\mathfrak{B}(a)(\overline{H_m(\overrightarrow{u_n})})\} \cup \{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_m})\} \cup E, S}$$

if $F \in \mathcal{V} - Var(\overrightarrow{x_k})$, $a \in \mathcal{F} \cup Var(\overrightarrow{u_n})$, $F \notin FVar(\lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_m}))$, and $\overline{H_m}$ new.

flex-flex-eq:

$$\frac{\{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.F(\overrightarrow{v_n})\} \cup E, S}{\{F =^? \lambda\overrightarrow{u_n}.H(\overrightarrow{z_r})\} \cup \{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.F(\overrightarrow{v_n})\} \cup E, S}$$

if $F \in \mathcal{V} - Var(\overrightarrow{x_k})$, $\{\overrightarrow{z_r}\} = \{z \mid z \in Var(\overrightarrow{u_n}), \overrightarrow{u_n}|_p = z = \overrightarrow{v_n}|_p\}$, and H new.

flex-flex-neq:

$$\frac{\{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.G(\overrightarrow{v_m})\} \cup E, S}{\{F =^? \lambda\overrightarrow{u_n}.H(\overrightarrow{z_r}), G =^? \lambda\overrightarrow{v_m}.H(\overrightarrow{z_r})\} \cup \{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.G(\overrightarrow{v_m})\} \cup E, S}$$

if $F, G \in \mathcal{V} - Var(\overrightarrow{x_k})$, $F \neq G$, $\{\overrightarrow{z_r}\} = Var(\overrightarrow{u_n}) \cap Var(\overrightarrow{v_m})$, and H new.

xflex-any:

$$\frac{\{\lambda\overrightarrow{x_k}.\mathfrak{B}(\pi_i^r(F(\overrightarrow{u_n})))(\overrightarrow{v_m}) =^? \lambda\overrightarrow{x_k}.t\} \cup E, S}{\{F =^? \lambda\overrightarrow{u_n}.\langle\overline{H_r(\overrightarrow{u_n})}\rangle\} \cup \{\lambda\overrightarrow{x_k}.\mathfrak{B}(\pi_i^r(F(\overrightarrow{u_n})))(\overrightarrow{v_m}) =^? \lambda\overrightarrow{x_k}.t\} \cup E, S}$$

if $F \in \mathcal{V} - Var(\overrightarrow{x_k})$ and $\overline{H_r}$ new.

Fig. 3. Unification for Higher-Order Product-Patterns

4.3 Soundness and Completeness of $\mathcal{PU}$

The next two lemmas are of rather technical nature. Their proofs are omitted due to lack of space.

Lemma 16. *Let θ be an arbitrary substitution, $t = \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_m})$ an xrigid term and $s = \lambda\overrightarrow{x_k}.\langle\overrightarrow{s_m}\rangle$ a product term. Then*
(a) $\theta(\lambda\overrightarrow{x_k}.t_i)\downarrow_P = \lambda\overrightarrow{x_k}.t_i'$ for all $i \in \{1, \ldots, m\}$ and $\theta(t)\downarrow_P = \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overrightarrow{t_m'})$
(b) $\theta(\lambda\overrightarrow{x_k}.s_i)\downarrow_P = \lambda\overrightarrow{x_k}.s_i'$ for all $i \in \{1, \ldots, m\}$ and $\theta(s)\downarrow_P = \lambda\overrightarrow{x_k}.\langle\overrightarrow{s_m'}\rangle$.

Lemma 17. *(a) Let* $\theta \in \mathcal{U}(\{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.F(\overrightarrow{v_n})\})$ *and* $\{\overline{z_r}\} = \{z \mid z \in Var(\overrightarrow{u_n}), \overrightarrow{u_n}|_p = z = \overrightarrow{v_n}|_p\}$. *Then* $\theta(\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n})) \xrightarrow{*}_{\beta_0} \lambda\overrightarrow{x_k}.t \xleftarrow{*}_{\beta_0} \theta(\lambda\overrightarrow{x_k}.F(\overrightarrow{v_n}))$ *and* $BVar(\lambda\overrightarrow{x_k}.t) \cap Var(\overrightarrow{x_k}) \subseteq \{\overline{z_r}\}$.
(b) Let $\theta \in \mathcal{U}(\{\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.G(\overrightarrow{v_m})\})$ *and* $\{\overline{z_r}\} = Var(\overrightarrow{u_n}) \cap Var(\overrightarrow{v_m})$. *Then* $\theta(\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n})) \xrightarrow{*}_{\beta_0} \lambda\overrightarrow{x_k}.t \xleftarrow{*}_{\beta_0} \theta(\lambda\overrightarrow{x_k}.G(\overrightarrow{v_m}))$ *and* $BVar(\lambda\overrightarrow{x_k}.t) \cap Var(\overrightarrow{x_k}) \subseteq \{\overline{z_r}\}$.

Now we are able to show that the set of unifiers essentially remains unchanged when performing an inference step.

Lemma 18. *Let* $(E, S) \vdash_{\mathcal{PU}} (E', S')$ *with one of the inferences* Delete, Decompose, Product-Decompose, *or* Eliminate. *Then* $\mathcal{U}(E \cup S) = \mathcal{U}(E' \cup S')$.

Proof. Delete is trivial. For Decompose and Product-Decompose, this is a corollary of Lem. 16. The proof for Eliminate is analogous to the proof found in [SG89].

Lemma 19. *Let* $(E, S) \vdash_{\mathcal{PU}} (E', S')$ *with one of the inferences* flex-xrigid, flex-flex-eq, flex-flex-neq, *or* xflex-any. *Then* $\mathcal{U}(E' \cup S') \subseteq \mathcal{U}(E \cup S)$ *and for each* $\theta \in \mathcal{U}(E \cup S)$ *exists a uniquely determined* θ_0 *such that* $(\theta \cup \theta_0) \in \mathcal{U}(E' \cup S')$, *and* $Dom(\theta_0)$ *consists of the newly introduced variables.*

Proof. For the first part consider $E' \supset E$ and $S' = S$. For the second part, we may assume with no loss of generality that the newly introduced variables are neither in the domain nor in the image of θ. We get the definition of θ_0 by an analysis of θ. In the case flex-xrigid, θ unifies $\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n}) =^? \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overline{t_m})$. Thus by Lem. 16(a), $\theta(\lambda\overrightarrow{x_k}.F(\overrightarrow{u_n})) \xrightarrow{*}_{\beta_0} \lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overline{r_m}) \xleftarrow{*}_{\lambda\times} \theta(\lambda\overrightarrow{x_k}.\mathfrak{B}(a)(\overline{t_m}))$, since we only consider product-patterns and θ is normalized and therefore $\theta(F) \approx_\alpha \lambda\overrightarrow{u_n}.\mathfrak{B}(a)(\overline{r_m})$. To unify the newly introduced equation, θ_0 has to be $\theta_0 = \{\overline{H_m \leftarrow \lambda\overrightarrow{u_n}.r_m}\}$. This is a sound extension of θ, since it uses the same binder and thus no free variable gets bound or vice versa. We get $\theta_0(\lambda\overrightarrow{u_n}.\mathfrak{B}(a)(\overline{H_m(\overrightarrow{u_n})})) = \lambda\overrightarrow{u_n}.\mathfrak{B}(a)(\overline{(\lambda\overrightarrow{u_n}.r_m)(\overrightarrow{u_n})}) \xrightarrow{*}_{\beta_0} \theta(F)$. The other equations in $E \cup S$ are already unified by θ, so they will be by $\theta \cup \theta_0$. For the other cases, analogous argumentations apply, using Lem. 17(a), Lem. 17(b), and Lem. 16(b) respectively.
Please note that for the argumentation the use of $\rightarrow_{\beta_0}$ was essential.

By a simple inductive argument, we get the following

Corollary 20. *Let* $(E, S) \vdash_{\mathcal{PU}} \ldots \vdash_{\mathcal{PU}} (E', S')$. *Then* $\mathcal{U}(E' \cup S') \subseteq \mathcal{U}(E \cup S)$ *and for each* $\theta \in \mathcal{U}(E \cup S)$ *exists a uniquely determined* θ_0 *such that* $(\theta \cup \theta_0) \in \mathcal{U}(E' \cup S')$, *and* $Dom(\theta_0)$ *consists of the newly introduced variables.*

Now, we are able to state the central results of this section:

Theorem 21 (Soundness). *Let* $(E, \varnothing) \vdash^*_{\mathcal{PU}} (\varnothing, S)$. *Then* $\sigma_S|_{FVar(E)} \in \mathcal{U}(E)$.

Proof. By Cor. 20, we have that $\sigma_S \in \mathcal{U}(E)$. But then, even the restricted substitution is a unifier, as only the free variables of E are affected.

Theorem 22 (Completeness). *Let* $\theta \in \mathcal{U}(E)$. *Then, there exists a finite derivation* $(E, \varnothing) \vdash_{\mathcal{PU}} \ldots \vdash_{\mathcal{PU}} (\varnothing, S)$ *and* $\theta \gtrsim_{\lambda\times} \sigma_S[FVar(E)]$.

Proof. The proof for the existence of a finite derivation leading to $(\varnothing, S)$ whenever $\mathcal{U}(E) \neq \varnothing$ will be given in the next section in a constructive manner. Then, by Cor. 20 there exists a θ_0 such that $(\theta \cup \theta_0) \in \mathcal{U}(S)$. Since S is in solved form, $(\theta \cup \theta_0) \gtrsim_{\lambda\times} \sigma_S$. But then $\theta \gtrsim_{\lambda\times} \sigma_S[Dom(\theta)]$ and furthermore $\theta \gtrsim_{\lambda\times} \sigma_S[FVar(E)]$.

Since in the above theorem the derivation is independent of the substitution θ, we get the important corollary:

Corollary 23 (Existence of mgu). *Let* $(E, \varnothing) \vdash_{\mathcal{PU}}^{*} (\varnothing, S)$. *Then* $\sigma_S|_{FVar(E)}$ *is a most general unifier of* E.

4.4 Adding Control

To ensure termination of the unification process we have to introduce some control on the application of the different inferences. One obvious reason for this is the formulation of the latter four inferences, which add some new equations. Also, to some equations different inferences may be applicable, e. g. Eliminate and flex-flex-neq. A third problem arises when the subproblems generated by flex-xrigid are not solved before the rest of the problem, which was already noted by Nipkow [Nip93]. His example was $\{F =^? c(G), G =^? c(F)\}$.

Therefore, we formulate an algorithm $\mathcal{A}$, the control strategy of which is sufficient to guarantee termination as we will show in the sequel.

Definition 24 Algorithm $\mathcal{A}$. Algorithm $\mathcal{A}$ is based on the inferences of system $\mathcal{PU}$. It organizes the set E as a *stack*. To each equation on top of that stack Delete and Eliminate are applied with priority. Then, only one of the other inferences may be applicable. If no inference is applicable $\mathcal{A}$ returns FAIL, otherwise it proceeds until E is empty. Then the solved part S is returned.

In the sequel, we will denote by $(E, S) \vdash_{\mathcal{A}} (E', S')$ inference steps according to that strategy. Considered as relations, $\vdash_{\mathcal{A}} \subset \vdash_{\mathcal{PU}}$.

That there is in fact no unifier, if no inference is applicable at some point, is shown by the following lemma:

Lemma 25. *Let* $(E, S) \vdash_{\mathcal{PU}} \ldots \vdash_{\mathcal{PU}} (\{s =^? t\} \cup E', S')$, *and to* $s =^? t$ *no inference is applicable. Then* $\mathcal{U}(E \cup S) = \varnothing$.

Proof. Since there is no inference applicable to $s =^? t$, one of the following cases must hold (which we get by inverting the conditions of the inferences).

- Both s and t are xrigid terms with different heads or different backbones.
- Failure of occur check (Eliminate or flex-xrigid)
- t is an xrigid term with head a which is a bound variable but does not occur in the arguments of the flex term s.

In each case, s and t are not unifiable and the result is established by Cor. 20.

To show that $\mathcal{A}$ will only generate finite derivations, we use some notions of complexity of equations, which is decreased by $\mathcal{A}$ w. r. t. a well-founded ordering.

Definition 26. The following sets describe the occurrences of backbones in terms (distinguished by their heads):

$B(t) = \{\mathfrak{B}(a) \mid t|_p = \mathfrak{B}(a)(\overline{t_m}) : \tau, \tau \in \mathcal{B}\}$

$B_f(t) = \{\mathfrak{B}(a) \mid \mathfrak{B}(a) \in B(t), a \in FVar(t)\} \qquad B_r(t) = B(t) - B_f(t)$

The multiset versions of these sets are denoted by $B^m(t)$, $B_f^m(t)$, and $B_r^m(t)$ respectively. To count the occurences of π-symbols in xflex backbones, we define:

$\mathrm{dp}_\pi(E) = \left|\bigcup_{t \in E} B_f(t)\right|_\pi$

whereby $|M|_\pi$ denotes the number of π-symbols occuring in M.

The *size* $|t|$ of a term t is inductively defined as $|a| = 1$ for $a \in \mathcal{F} \cup \mathcal{V}$ and

$|\lambda p.t| = |t| \qquad |\langle\overline{t_m}\rangle| = 1 + |t_1| + \ldots + |t_m|$

$|s\ t| = |s| + |t| \qquad |\pi_i^n(t)| = 1 + |t|$

We call a substitution θ *size increasing*, if $|(\theta(t))\downarrow_P| > |t\downarrow_P|$ for some term t.

The *complexity* $\mathbf{C}(E)$ of a finite system of equations E is defined as the triple $(\mathrm{dp}_\pi(E), |FVar(E)|, |E|)$, where $\mathrm{dp}_\pi(E)$ is defined as above, $|FVar(E)|$ denotes the number of distinct free variables of E, and $|E|$ is the size of E, i. e. the sum of the sizes of its terms. These triples are compared by $\succ$, the lexicographical combination on triples of the standard ordering $>$ on the naturals.

The first four inferences do directly decrease the complexity:

Lemma 27. *Let* $(E, S) \vdash_{\mathcal{PU}} (E', S')$ *with inferences* Delete,Decompose,Product-Decompose, *or* Eliminate. *Then* $\mathbf{C}(E) \succ \mathbf{C}(E')$.

Proof. Delete may decrease $\mathrm{dp}_\pi(\cdot)$ or $|FVar(\cdot)|$, but surely decreases $|\cdot|$. Decompose and Product-Decompose do not alter $\mathrm{dp}_\pi(\cdot)$ or $|FVar(\cdot)|$, but decrease $|\cdot|$ by at least 2. Eliminate clearly decreases $|FVar(\cdot)|$ by 1, and may decrease but never increases $\mathrm{dp}_\pi(\cdot)$ even if xflex terms are substituted. This is due to the use of the *set* of *backbones*, which is different from the set of xflex terms. Thus, the possible duplicating effect of the substitution followed by implicit reductions, which may lead to different argument structures, is harmless.

Since the other inferences clearly increase the complexity by introducing new equations, we have to consider sequences of inferences determined by the control strategy of algorithm $\mathcal{A}$.

Lemma 28. *Let* $(E, S) \vdash_{\mathcal{PU}} (E', S')$ *with inferences* flex-xrigid, flex-flex-eq, flex-flex-neq, *or* xflex-any. *Then* $(E', S') \vdash_{\mathcal{A}}^* (E'', S'')$ *such that* $\mathbf{C}(E) \succ \mathbf{C}(E'')$.

Proof. After applying flex-flex-eq, the next step is Eliminate followed by Delete, because the considered equation is unified by the introduced approximation. In this sequence $\mathrm{dp}_\pi(\cdot)$ remains unchanged, $|FVar(\cdot)|$ is constant (F is replaced by H), and as the substitution is not size increasing, the Delete-step decreases $|\cdot|$. The inference flex-flex-neq is followed by two Eliminate-steps and a Delete, since the original equation is unified. As above, $\mathrm{dp}_\pi(\cdot)$ remains unchanged, but

Inference	$\mathrm{dp}_\pi(E)$	$\lvert FVar(E)\rvert$	$\lvert E\rvert$
Delete	$\geq$	$\geq$	$>$
Decompose	$=$	$=$	$>$
Product-Decompose	$=$	$=$	$>$
Eliminate	$\geq$	$>$	$*$

Sequ. of Inf.	$\mathrm{dp}_\pi(E)$	$\lvert FVar(E)\rvert$	$\lvert E\rvert$
flex-flex-eq'	$=$	$=$	$>$
flex-flex-neq'	$=$	$>$	$*$
xflex-any'	$>$	$*$	$*$
flex-xrigid'	$>$	$*$	$*$
	$=$	$>$	$*$

Fig. 4. Decreasing the complexity ($*$ means "don't care")

$|FVar(\cdot)|$ is decreased, since the two variables F and G are replaced by the new H. In the case of xflex-any, the implicitely performed π-reductions during the following Eliminate will decrease $\mathrm{dp}_\pi(\cdot)$.

The case flex-xrigid is more complicated. Let $E = \{s =^? t\} \cup E_0$ with $s = \lambda\overleftrightarrow{x_k}.F(\overleftrightarrow{u_n})$ and $t = \lambda\overleftrightarrow{x_k}.\mathfrak{B}(a)(\overleftrightarrow{t_m})$. After performing flex-xrigid and Eliminate, a Decompose takes place. Then we have the following situation:

$$(\{\lambda\overleftrightarrow{x_k}.H_1(\overleftrightarrow{u_n}) =^? \lambda\overleftrightarrow{x_k}.t_1, \ldots, \lambda\overleftrightarrow{x_k}.H_m(\overleftrightarrow{u_n}) =^? \lambda\overleftrightarrow{x_k}.t_m\} \cup \sigma_0(E_0), \hat{S})$$

Assuming the system is unifiable, i. e. algorithm $\mathcal{A}$ does not stop with FAIL, the inferences performed by $\mathcal{A}$ for the i-th subproblem are equivalent to

$$(\{\lambda\overleftrightarrow{x_k}.H_i(\overleftrightarrow{u_n}) =^? \sigma_{S_{i-1}}(\cdots(\sigma_{S_1}(\lambda\overleftrightarrow{x_k}.t_i))\cdots)\}, \varnothing) \vdash_{\mathcal{A}}^* (\varnothing, S_i)$$

which eventually leads to the situation $E'' := \sigma(E_0)$ with $\sigma = \sigma_{S_m} \circ \cdots \circ \sigma_{S_1} \circ \sigma_0$.

Before we analyze $\mathbf{C}(E'')$ we have to assure that the subproblems really have finite derivations. Since all the other inferences decrease the complexity, the only chance to get non-termination is the use of flex-xrigid itself. Here, $B_r^m(t)$ comes into play, which in this situation describes the chances to apply flex-xrigid in the derivation. We have $B_r^m(t) \supset B_r^m(\lambda\overleftrightarrow{x_k}.t_j)$ for each $j \in \{1, \ldots, m\}$ and a careful analysis of the substitutions σ_{S_i} reveals, that they do not change $B_r^m(\lambda\overleftrightarrow{x_k}.t_j)$.

It remains to compare $\mathbf{C}(E)$ with $\mathbf{C}(E'')$. We have to distinguish two cases: If $\mathrm{dp}_\pi(t) > 0$ at least one xflex-any-inference will be performed, because t gets fully decomposed. This decreases $\mathrm{dp}_\pi(E)$ and we are done. If, however, $\mathrm{dp}_\pi(t) = 0$ σ does not change $\mathrm{dp}_\pi(\cdot)$. But then $|\mathcal{I}(\sigma|_{FVar(t)})| = |FVar(t)|$ and furthermore $\mathcal{I}(\sigma|_F) = \mathcal{I}(\sigma|_{FVar(t)})$. Therefore $|\mathcal{I}(\sigma|_{FVar(E)})| = |FVar(s = t)| - 1$ which establishes $|FVar(\{s = t\} \cup E_0)| > |FVar(E'')|$.

A summarized version of the above argumentation is shown in Fig. 4. The primed versions denote the sequences which lead to E'' in Lem. 28.

Now, $\succ$ is well-founded and therefore $\mathcal{A}$ will only generate finite derivations to return either FAIL when no unifier exists or a solved form S. This establishes the missing part of the proof of Thm. 22 and leads to the following

Theorem 29. *The unification problem for higher-order product-patterns is decidable. In the positive case, a most general unifier can be computed.*

5 Extension: Integrating the Terminal Type

Adding the terminal[3] type $\mathbf{T}$ is motivated by the close relationship between lambda-calculus and category theory. This type represents the terminal object (hence its name) in *cartesian closed categories* and consists of just one element, denoted by $*$, which may be seen as the nullary tuple. The following extensional axiom describes this property:

$$top: \quad t =_{top} * \qquad \text{if } t : \mathbf{T}$$

In contrast to the other extensional axioms, this equation may only be oriented from left to right. As is well known [DC95], there are no problems with terms in $\lambda\times$-long form [Aka93, JG95], but $\rightarrow_{top}$ destroys confluence with $\rightarrow_{\eta\, sp}$ [LS86]. This is important to our approach, since we require η *sp*-short forms for the arguments of free variables.

To overcome these obstacles, we adopt the approach of [DC95, Chap. 2.3] to our pattern calculus. First, for all types $\tau \in Iso(\mathbf{T})$, i.e. the types which are isomorphic to $\mathbf{T}$ such as $\hat{\tau} \rightarrow \mathbf{T}$ or $\mathbf{T} \times \mathbf{T}$, we introduce a generic representative $*_\tau$. The relation *top* has to be stated in a general version:

$$gentop: \quad t \rightarrow_{top} *_\tau \qquad \text{if } t : \tau,\ t \neq *_\tau, \text{ and } \tau \in Iso(\mathbf{T})$$

Then, we extend Def. 2 to allow the (multiple) occurrence of some $*_\tau$ in abstraction patterns and adjoin $\rightarrow_{top}$ to $\rightarrow_\alpha$. This resolves the confluence problem between $\rightarrow_\eta$ and $\rightarrow_{top}$.

To get full confluence, we have to extend $\rightarrow_{sp}$ to tuples with some components with a type $\tau_i \in Iso(\mathbf{T})$ and at least one with $\tau_j \notin Iso(\mathbf{T})$.

$$sp_{top}: \quad \langle \ldots *_{\tau_i} \ldots \pi_j^n(t) \ldots \rangle \rightarrow_{sp} t$$

This re-establishes the uniqueness of the pattern normal form.

Since we have a simply typed framework, the occurrences of $*_\tau$ are covered by Delete resp. by the definitions of the sets $\{\overline{z_\tau}\}$ in flex-flex-eq and flex-flex-neq. This means that neither the inference system nor the proofs have to be changed by this extension besides the definition of the pattern normal form and that $|*_\tau| = 1$. Especially, the proofs of Sect. 4.3 and Sect. 4.4 literally remain the same.

6 Conclusion and Future Work

Our approach is characterized by a successful combination of several generalizations of known concepts. Binding of variables is generalized to the binding of abstraction patterns. Flex/rigid terms are generalized to xflex/xrigid terms due to the introduction of the concept of a backbone. Higher-order patterns are generalized to higher-order product-patterns by allowing abstraction patterns as arguments of xflex terms. The combination of these concepts supports the generalization of β_0-reduction to a calculus with integrated products. The availability of this highly restricted kind of β-reduction permits the formulation of elegant inference rules.

[3] This is the *unit* type in functional programming languages such as *Haskell* or *ML*.

The elegance of our system becomes evident when comparing it to the work of Duggan [Dug93]. He deals with a richer type language than we do. However, when restricted to product and function space types his approach still needs a much more complicated framework than ours. The complications mostly arise from the translation of the usual lambda calculus in his locator calculus. Due to the conceptual clearness of the inference system $\mathcal{PU}$ and since it specializes to that presented by Nipkow [Nip91] when terms without product types are unified we think that our approach is more natural than that of Duggan's. Moreover, as outlined in section 5 the integration of the terminal type is possible by extending the definitions of abstraction patterns and $\downarrow_P$ only which demonstrates the modularity of our approach. Therefore, we are confident that we will soon present an extension to second order polymorphic types.

References

[Aka93] Y. Akama. On Mints' reductions for ccc-Calculus. In *Typed Lambda Calculi and Applications*, volume 664 of *LNCS*, pages 1–12, 1993.

[DC95] R. Di Cosmo. *Isomorphisms of Types.* Birkhäuser, 1995.

[DCK93] R. Di Cosmo and D. Kesner. A confluent reduction for the extensional typed λ-calculus with pairs, sums, recursion, and terminal object. In A. Lingas et al., editors, *ICALP*, volume 697 of *LNCS*, pages 645–656, 1993.

[Dug93] D. Duggan. Unification with Extended Patterns. Technical Report CS-93-37, University of Waterloo, 1993. To appear in *Theoretical Computer Science.*

[Gol81] W. D. Goldfarb. The undecidability of the second-order unification problem. *Theoretical Computer Science*, 13:225–230, 1981.

[Hue75] G. P. Huet. A Unification Algorithm for Typed λ-Calculus. *Theoretical Computer Science*, 1:27–57, 1975.

[JG95] C. B. Jay and N. Ghani. The virtues of eta-expansion. *J. Functional Programming*, 5(2):135–154, April 1995.

[Joh94] T. Johnsson. Fold-unfold transformations on state monadic interpreters. In K. Hammond et al., editors, *Functional programming, Glasgow*, Workshops in Computing. Springer-Verlag, 1994.

[Kes94] D. Kesner. Reasoning about Layered, Wildcard and Product Patterns. In G. Levi and M. Rodríguez-Artalejo, editors, *Algebraic and Logic Programming*, volume 850 of *LNCS*, pages 253–268, 1994.

[LS86] J. Lambek and P. J. Scott. *Introduction to higher order categorical logic.* Cambridge University Press, 1986.

[Mil91] D. Miller. A Logic Programming Language with Lambda-Abstraction, Function Variables, and Simple Unification. *J. Logic Comp.*, 1(4):497–536, 1991.

[Nip91] T. Nipkow. Higher-order critical pairs. In *Proc. sixth annual IEEE Symposium on Logic in Computer Science*, pages 342–349, 1991.

[Nip93] T. Nipkow. Functional unification of higher-order patterns. In *Proc. eighth annual IEEE Symposium on Logic in Computer Science*, pages 64–74, 1993.

[Pot81] G. Pottinger. The Church Rosser Theorem for the Typed lambda-calculus with Surjective Pairing. *Notre Dame J. of Formal Logic*, 22(3):264–268, 1981.

[SG89] W. Snyder and J. Gallier. Higher-Order Unification Revisited: Complete Sets of Transformations. *Journal of Symbolic Computation*, 8:101–140, 1989.